

Projet OpenGL

Malek.Bengougam@esiee.fr

Projet final à faire en groupe de 3 (voire 2) en OpenGL 2.x, 3.x ou 4.x au choix

Objectif : être capable d’afficher une scène 3D composée de plusieurs objets et pouvoir naviguer dans la scène.

- Par exemple, un objet avec une couleur simple, un autre avec une texture, un autre avec une environment map, juste du Lambert, d’autres Phong etc...

Partie 1 : Affichage

1.a - Chargement de modèle 3D (*cf. Annexe A plus bas*)

Etre capable de charger des objets au format OBJ en utilisant la bibliothèque TinyOBJLoader, en gérant les matériaux (couleurs ambiantes, diffuses, spéculaires)
<https://github.com/tinyobjloader/tinyobjloader>

1.b - Illumination directe

Avoir de l’illumination avec l’équation d’illumination de Phong (avec un spéculaire de Phong ou Blinn-Phong) combinant textures et matériaux issus du fichier OBJ.

1.c - Illumination indirecte

Les contributions diffuses (Lambert) et spéculaires (Phong) que nous avons vues jusqu’à présent sont des contributions “directes”.

La contribution ambiante est théoriquement une contribution indirecte, et est en fait également subdivisées en composantes diffuses et spéculaires dites “indirectes”.

On peut donc changer l’équation d’illumination de Phong de

Couleur Finale = Ambient + ForEach_LumiereDirecte(Diffuse + Spéculaire)
en

Couleur Finale = Diffuse_{Indirect} + Speculaire_{Indirect} +
ForEach_LumiereDirecte(Diffuse_{Direct} + Speculaire_{Direct})

Utilisez une ou plusieurs de ces méthodes pour simuler la partie ambiante.

- ambiante hémisphérique (diffuse)
- “environment mapping” (spéculaire)
- Il est possible d’utiliser l’environnement mapping pour la partie diffuse.
Dans ce cas il faut utiliser directement la normale du fragment.

1.d - rendu hors écran

Votre rendu principal s’effectue hors écran via un FBO et est recopié dans le backbuffer

Note : dans tous les cas, attention à bien gérer les conversions gamma (sRGB) vers linéaire et vice-versa !

Partie 2 : Navigation

2.a - Les objets doivent être placés à des positions différentes dans la scene avec des translations, rotations, scale etc... propres à chaque objet.

2.b Utilisez un UBO pour transférer les matrices projection+camera à tous vos shaders et pourquoi pas un autre pour la transform (world matrix voire d'autres paramètres spécifiques à une instance).

Le plus important est d'avoir une caméra qui peut se déplacer dans la scene et/ou orbiter autour d'un objet.

Partie 3 : Options

3.a - Effectuez un ou plusieurs post-traitement à l'aide FBO et de shaders.

3.b - Utilisez une technique d' "instancing" hardware pour passer les données spécifiques aux instances (world matrix, couleurs...).

3.c - Affichez une skybox en utilisant la technique des cubemap

3.d - Utilisez un Compute Shader pour générer une texture procédurale

3.e - Implémenter une interface graphique à l'aide d'IMGUI

<https://github.com/ocornut/imgui>

3.f - Parfois un objet est illuminé par des sources lumineuses derrière l'objet mais face à la caméra. Ce qui produit un effet d'illumination arrière (back-lighting) qui illumine les contours d'un objet. Cet effet est souvent (un peu à tort) appelé effet Fresnel.

Programmez un shader qui implémente cet effet (spécifiquement ou combiné à Phong)

3.g - Théoriquement, les composantes diffuses et spéculaires (directes ou indirectes) doivent se compenser afin de ne pas réfléchir plus d'énergie que la surface n'en a reçu.

Implémentez une forme approximative de l'équation de Fresnel (celle de Christophe Schlick étant la plus répandue) afin de balancer / mixer plus correctement le diffuse et le spéculaire.

Annexes

A. Affichage des objets 3D

La plupart du temps ces données sont fournies directement par le logiciel de modélisation.

On va utiliser la librairie `tiny_obj_loader` afin de charger des fichiers de type `obj`.

<https://github.com/tinyobjloader/tinyobjloader>

Il suffit d'ajouter les fichiers `tiny_obj_loader.h` et `tiny_obj_loader.cc` à votre projet.

La structure définissant la variable `mesh` contient les informations de position, coordonnées de texture et normales (si existantes) ainsi que la liste des indices pour la forme actuelle.

La particularité du format `OBJ` réside dans le format de description des faces :

Une face est composée de triplets d'indices : en effet les positions, normales et coordonnées de textures sont stockées dans des tableaux différents, et pour des raisons d'optimisation, les tableaux peuvent avoir une taille différente !

On a donc 3 indices, 1 pour les positions, 1 pour les coordonnées de texture et 1 pour les normales.

Comme `OpenGL` n'est pas capable de gérer 3 tableaux d'indices il faut donc recréer les vertices et éventuellement les indices pour être compatible avec `OpenGL`.

Voir l'exemple de récupération des indices à partir des « shapes » `TinyObjLoader` dans le `README.md`.

Note : il est important de s'assurer que votre modèle 3D est bien triangularisé (composé de triangles et pas de quads ou de polygones). Alternativement, `TinyObjLoader` offre une option pour forcer la triangularisation.

La bibliothèque gère également les fichiers `MTL` qui contiennent les informations de matériaux : en premier lieu on a les textures à utiliser pour les composantes ambiantes, diffuses et spéculaires.

Le format supporte également des valeurs de couleurs pour chacune de ces composantes.